

5) Answer

- If destination reached $\rightarrow$ return cost.
- Else $\rightarrow$ return -1

Code:

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
int findCheapestPrice (int n, vector<vector<int>>> flights, int src, int dst, int k) {
```

```
    vector<vector<pair<int, int>>> adj(n);
```

```
    for (auto f : flights) {
```

```
        adj[f[0]].push_back({f[1], f[2]});
```

```
    }
```

```
    queue<tuple<int, int, int>> q;
```

```
    q.push({src, 0, 0});
```

```
    vector<int> dist(n, INT_MAX);
```

```
    dist[src] = 0;
```

```
    while (!q.empty()) {
```

```
        auto [node, cost, stops] = q.front();
```

```
        q.pop();
```

```
        if (stops > k) continue;
```

```
        for (auto [next, price] : adj[node]) {
```

```
            if (cost + price < dist[next]) {
```

```
                dist[next] = cost + price;
```

//_

```
q.push (Enext, dist[next], steps + 1);  
}  
}
```

```
return dist[dst] == INT_MAX ? -1 : dist[dst];  
}
```

Time complexity

$O(K \times E)$

↑
max no. of steps allowed.

Space complexity

$O(V + E)$